

Laboratory for Bio- and Nano-Instrumentation

# AFM Controller LabVIEW Application

General Documentation

LBNI & Sci-Consulting  
2020-03-26

## Contents

|                                                                             |           |
|-----------------------------------------------------------------------------|-----------|
| <b>Introduction</b>                                                         | <b>15</b> |
| <b>Architecture: Overview</b>                                               | <b>16</b> |
| <b>Architecture: AFM Head and Scanner</b>                                   | <b>17</b> |
| AFM Head                                                                    | 17        |
| AFM Scanner                                                                 | 17        |
| <b>Architecture: Controller</b>                                             | <b>18</b> |
| X-Y Scanner                                                                 | 18        |
| Z Closed loop                                                               | 18        |
| Data Acquisition & Excitation                                               | 18        |
| Host Data Averaging                                                         | 18        |
| Coarse Z                                                                    | 18        |
| <b>Architecture: Host PC</b>                                                | <b>19</b> |
| Z Controller                                                                | 19        |
| Scan Parameters                                                             | 19        |
| Excitation                                                                  | 19        |
| Monitor                                                                     | 19        |
| <b>Implementation: Overview</b>                                             | <b>20</b> |
| <b>Implementation: Processes and loops</b>                                  | <b>22</b> |
| loop.001 : Multiplexer Parameters                                           | 22        |
| loop.002 : Multiplexer Output                                               | 22        |
| loop.003 : Step Motor Enable & Direction Output                             | 23        |
| loop.004 : Stepper Motor Driver Interface                                   | 23        |
| loop.005 : Fast ADC acquisition and DAC generation                          | 23        |
| loop.006 : Host data transfer interaction wave generator and memory control | 24        |
| loop.007 : z Watchdog for auto-approach                                     | 25        |
| loop.008 : Local wave recording hypervisor for ORT                          | 25        |
| loop.009 : Host data averaging                                              | 25        |
| loop.010 : PID calculation and height control                               | 26        |
| loop.011 : Slow Height generation (output)                                  | 26        |
| loop.012 : SCTL to hasten loop.011 execution                                | 27        |

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| loop.013 : Slow fit calculation                                            | 27 |
| loop.014 : Main process variable calculation                               | 27 |
| loop.015 : Input module acquisition and excitation output                  | 27 |
| loop.016 : Populate memory items for feedback from host                    | 27 |
| loop.017 : SICM                                                            | 28 |
| loop.018 : End of transmission acknowledge                                 | 28 |
| loop.019 : Transfer interaction data to host                               | 28 |
| loop.020 : Transfer background to host                                     | 28 |
| loop.021 : Population of many registers and AI voltage range               | 29 |
| loop.022 : Host data transfer main                                         | 29 |
| loop.023 : Retract on background correct to get non-interaction background | 29 |
| loop.024 : Output the Step Motor state                                     | 30 |
| loop.025 : Write a Sawtooth Waveform to 3 FIFOs (Stick-Slip)               | 30 |
| loop.026 : Triggers End of Line / Frame                                    | 30 |
| loop.027 : Slow host-controlled signal generator                           | 30 |
| loop.028 : Invert and blank inputs                                         | 30 |
| loop.029 : Slow Analog inputs                                              | 30 |
| loop.030 : Output averaging and pixel synchronous output                   | 30 |
| loop.031 : Output to host                                                  | 30 |
| loop.032 : Input recording and passover to averaging                       | 30 |
| loop.033 : Pixel Sync                                                      | 30 |
| loop.034 : Automatic background correct request                            | 31 |
| loop.035 : X-Y Physical Output                                             | 31 |
| loop.036 : SCTL to hasten loop.035 execution                               | 31 |
| loop.037 : Pixel Events Aux Sync                                           | 31 |
| loop.038 : Monitor Clear Pixel Event                                       | 31 |
| loop.039 : HSDC recording                                                  | 31 |
| loop.040 : Monitor Pixel Sync Event                                        | 31 |
| loop.041 : Read and write HSDC storage                                     | 31 |
| loop.042 : Scan Engine                                                     | 32 |
| loop.043 : Memory write access for Scan Engine                             | 32 |
| loop.044 : ITC Clear Pixel and Pixel Event Sync                            | 32 |
| loop.045 : Monitor Pixel Sync Aux Event                                    | 32 |

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>Implementation: Functions and VIs</b>                | <b>33</b> |
| sicm_main.vi : SICM Main                                | 33        |
| ort_fit_calcSlopeDissipation.vi : Slow Fit              | 33        |
| invert_and_blank_caseStruct.vi : Rock N Sock            | 33        |
| ort_find_max.vi                                         | 33        |
| ort_fit_start.vi                                        | 33        |
| ort_fit_calcIntermediate.vi                             | 33        |
| ort_bgc_main.vi                                         | 33        |
| circularCounter_v2.vi                                   | 33        |
| inputFilter_aio.vi                                      | 33        |
| lwr_main.vi : Inter Average to BRAM                     | 33        |
| intWaveMA.vi                                            | 33        |
| pid_main_v2.vi                                          | 33        |
| sid_main.vi                                             | 33        |
| rmp_main.vi                                             | 33        |
| flt_sos4xblock.vi                                       | 33        |
| flt_sos8xblock.vi                                       | 33        |
| hdt_int_generator.vi                                    | 34        |
| hdt_main_averageChannels.vi                             | 34        |
| ort_bgc_hypervisor.vi                                   | 34        |
| lwr_hypervisor.vi                                       | 34        |
| hsdc_hypervisor.vi                                      | 34        |
| se_main.vi : Scan Engine Main                           | 34        |
| se_sid_main.vi : Scan Engine System Identification Main | 34        |
| pid_XY.vi                                               | 34        |
| hdt_aux_averageChannels.vi                              | 34        |
| waveformGerneratorStickSlip_v3.vi                       | 34        |
| <b>Implementation: Items</b>                            | <b>35</b> |
| AI.voltage                                              | 35        |
| aio-hdt.aux                                             | 35        |
| aio-hdt.x                                               | 35        |
| aio-ort.t.endOfCycle                                    | 35        |
| aio.excitation                                          | 35        |

|                            |    |
|----------------------------|----|
| aio.flt.bypass             | 35 |
| aio.flt.p.anot4x           | 35 |
| aio.flt.p.b4x              | 36 |
| aio.i.samplingDecimation   | 36 |
| aio.i.slowInputRate        | 36 |
| aio.m.drive                | 36 |
| aio.o.bypassDither         | 36 |
| aio.o.disableExc           | 36 |
| aio.o.fastInput            | 36 |
| aio.o.heightLimits         | 36 |
| aio.o.inputs               | 37 |
| aio.o.invert2Inputs        | 37 |
| aio.o.invertHeight         | 37 |
| aio.o.invertInputs         | 37 |
| aio.o.invertOutputs        | 37 |
| aio.o.outputs              | 37 |
| aio.p.fastADCclockdec      | 37 |
| aio.s.HSaux                | 37 |
| aio.s.clock                | 38 |
| aio.s.fastHoriz            | 38 |
| aio.s.fastVertical         | 38 |
| aio.s.ortSubstDeflection   | 38 |
| aio.s.slowActualFBOutRate  | 38 |
| aio.s.slowActualInputRate  | 38 |
| aio.s.slowActualOutputRate | 38 |
| aio.s.slowHoriz            | 38 |
| aio.s.slowVertical         | 39 |
| aio.s.sum                  | 39 |
| aio.t.HSauxIn              | 39 |
| aio.t.fastDrive            | 39 |
| aio.t.fastVert             | 39 |
| aio.t.indexedValue         | 39 |
| aio.t.input                | 39 |

|                         |    |
|-------------------------|----|
| aio.t.slowHeightDrive   | 40 |
| aio.t.slowHorizVert     | 40 |
| aio.wfm.enable          | 40 |
| aio.wfm.x               | 40 |
| aio.wfm.y               | 40 |
| aio.wfm.z               | 40 |
| aoi.i.slowInputRate     | 40 |
| clocksOut               | 40 |
| dio.i.dataIn            | 41 |
| dio.i.multiplexerDataIn | 41 |
| dio.p.multiplexeCLK     | 41 |
| dio.p.multiplexerOE     | 41 |
| dio.p.multiplexerReset  | 41 |
| dio.p.multiplexerSync   | 41 |
| dio.p.multiplexerclock  | 41 |
| enableFeedback          | 41 |
| fb-aio.t.heightInt      | 42 |
| fb-hdt.int.t.procVar    | 42 |
| fb-hdt.t.bwdSlope       | 42 |
| fb-hdt.t.dissipation    | 42 |
| fb-hdt.t.error          | 42 |
| fb-hdt.t.height         | 42 |
| fb-hdt.t.minBwd         | 42 |
| fb-rmp.int.t.procVar    | 42 |
| fb-sicm.t.current       | 43 |
| fb.altSetpoint          | 43 |
| fb.enable               | 43 |
| fb.i.raw                | 43 |
| fb.i.rescaled           | 43 |
| fb.i.source             | 43 |
| fb.liftOff              | 43 |
| fb.loff.p.params        | 43 |
| fb.mot.m.motion         | 44 |

|                                    |    |
|------------------------------------|----|
| fb.mot.motion                      | 44 |
| fb.mot.p.params                    | 44 |
| fb.mot.p.params.enableManualMotion | 44 |
| fb.mot.s.running                   | 44 |
| fb.o.height                        | 44 |
| fb.p.allowSetpointSwitch           | 44 |
| fb.p.enable                        | 44 |
| fb.p.limits                        | 45 |
| fb.p.limits.lowerLimit             | 45 |
| fb.p.limits.upperLimit             | 45 |
| fb.p.params                        | 45 |
| fb.p.params.setPoint               | 45 |
| fb.p.zWatchDogTreshold             | 45 |
| fb.s.atLimit                       | 45 |
| fb.t.procVar                       | 45 |
| fb.z.WatchdogActive                | 46 |
| fb.zWatchdogArm                    | 46 |
| feedbackbParams                    | 46 |
| filt.bypassPI                      | 46 |
| flt.bypass1                        | 46 |
| flt.bypass2                        | 46 |
| flt.bypassPI                       | 46 |
| flt.i.bypassSource                 | 47 |
| flt.o.height                       | 47 |
| flt.o.hold                         | 47 |
| flt.p.anot4x                       | 47 |
| flt.p.anot8x                       | 47 |
| flt.p.b4x                          | 47 |
| flt.p.b8x                          | 47 |
| flt.ss.p.enable                    | 47 |
| flt.ss.p.rate                      | 48 |
| hdt.aux.o.enable                   | 48 |
| hdt.debugStream                    | 48 |

|                                               |    |
|-----------------------------------------------|----|
| hdt.int.p.memWriteSource                      | 48 |
| hdt.int.p.readParams                          | 48 |
| hdt.int.p.readParams.hdt.int.p.readLength     | 48 |
| hdt.int.p.readParams.hdt.int.s.transferSource | 48 |
| hdt.int.s.clearDataLost                       | 48 |
| hdt.int.s.dataLost                            | 49 |
| hdt.int.s.debug                               | 49 |
| hdt.int.staging                               | 49 |
| hdt.int.storage                               | 49 |
| hdt.int.t.transferDone                        | 49 |
| hdt.mainOutputStreamSelect                    | 49 |
| hdt.o.enable                                  | 49 |
| hdt.o.overflow                                | 49 |
| hdt.o.ticksPerPixel                           | 50 |
| hdt.outputEnable                              | 50 |
| hdt.p.mainAverageEnabled                      | 50 |
| hdt.t.mainStreamClear                         | 50 |
| hdt.t2h.auxStream                             | 50 |
| hdt.t2h.interaction                           | 50 |
| hdt.t2h.mainStream                            | 50 |
| hxdc.o.address                                | 50 |
| hxdc.o.read                                   | 51 |
| hxdc.p.points                                 | 51 |
| hxdc.p.start                                  | 51 |
| hxdc.p.substBgnd                              | 51 |
| hxdc.running                                  | 51 |
| hxdc.storage                                  | 51 |
| hxdc.t..done                                  | 51 |
| hxdc.t.start                                  | 51 |
| inBundleAux                                   | 52 |
| itc-hdt.t.pixelEventAux                       | 52 |
| itc.p.allowBGC                                | 52 |
| its.p.allowBGC                                | 52 |

|          |    |
|----------|----|
| loop.001 | 52 |
| loop.002 | 52 |
| loop.003 | 52 |
| loop.004 | 52 |
| loop.005 | 52 |
| loop.006 | 53 |
| loop.007 | 53 |
| loop.008 | 53 |
| loop.009 | 53 |
| loop.010 | 53 |
| loop.011 | 53 |
| loop.012 | 53 |
| loop.013 | 53 |
| loop.014 | 54 |
| loop.015 | 54 |
| loop.016 | 54 |
| loop.017 | 54 |
| loop.018 | 54 |
| loop.019 | 54 |
| loop.020 | 54 |
| loop.021 | 54 |
| loop.022 | 55 |
| loop.023 | 55 |
| loop.024 | 55 |
| loop.025 | 55 |
| loop.026 | 55 |
| loop.027 | 55 |
| loop.028 | 55 |
| loop.029 | 55 |
| loop.030 | 56 |
| loop.031 | 56 |
| loop.032 | 56 |
| loop.033 | 56 |

|                             |    |
|-----------------------------|----|
| loop.034                    | 56 |
| loop.035                    | 56 |
| loop.036                    | 56 |
| loop.037                    | 56 |
| loop.038                    | 57 |
| loop.039                    | 57 |
| loop.040                    | 57 |
| loop.041                    | 57 |
| loop.042                    | 57 |
| loop.043                    | 57 |
| loop.044                    | 57 |
| loop.045                    | 57 |
| lwr.o.readData              | 58 |
| lwr.p.avgCnt                | 58 |
| lwr.p.readAddress           | 58 |
| lwr.p.start                 | 58 |
| lwr.sum                     | 58 |
| lwr.t.done                  | 58 |
| lwr.t.start                 | 58 |
| mMot.running                | 59 |
| moveToWave                  | 59 |
| ort-hdt.int.t.fcBgnd        | 59 |
| ort-hdt.int.t.fcBgndRelease | 59 |
| ort-ort.fit.t.start         | 59 |
| ort.bgc.average             | 59 |
| ort.bgc.backgroundOK        | 59 |
| ort.bgc.p.bgCntP2           | 59 |
| ort.bgc.p.scannerTc         | 60 |
| ort.bgc.p.start             | 60 |
| ort.bgc.p.waitTicks         | 60 |
| ort.bgc.requestType         | 60 |
| ort.bgc.sum                 | 60 |
| ort.bgc.t.start             | 60 |

|                                       |    |
|---------------------------------------|----|
| ort.fit.accSums                       | 60 |
| ort.fit.accWeights                    | 60 |
| ort.fit.coefficients                  | 61 |
| ort.fit.m.coefficients                | 61 |
| ort.fit.m.weights                     | 61 |
| ort.fit.p.params                      | 61 |
| ort.fit.t.integralParts               | 61 |
| ort.fit.waveBwd                       | 61 |
| ort.fit.waveFwd                       | 61 |
| ort.interactionFiltered               | 61 |
| ort.o.interactionMax                  | 62 |
| ort.o.maxIndex                        | 62 |
| ort.o.piezoMode                       | 62 |
| ort.p.fcAvg                           | 62 |
| ort.p.fcAvgEnable                     | 62 |
| ort.p.methodParams                    | 62 |
| ort.p.methodParams.ort.p.syncPoint    | 62 |
| ort.p.methodParams.ort.p.useSyncPoint | 62 |
| ort.p.methodParams.ort.p.winSizeP2    | 63 |
| ort.p.waveSize                        | 63 |
| ort.waveSize                          | 63 |
| outAuxIn                              | 63 |
| outBundle                             | 63 |
| outBundleAux                          | 63 |
| rmp-hdt.aux.t.pixelEvent              | 63 |
| rmp-hdt.t.pixelEvent                  | 63 |
| rmp.p.action                          | 64 |
| rmp.p.params                          | 64 |
| rmp.p.reset                           | 64 |
| rmp.p.running                         | 64 |
| rmp.p.triggerParams                   | 64 |
| rmp.p.useIntHeightAsStart             | 64 |
| rmp.running                           | 64 |

|                               |    |
|-------------------------------|----|
| rmp.s.running                 | 64 |
| se-dio.t.pixelEvent           | 65 |
| se-hdt.t.pixelEvent           | 65 |
| se-hdt.t.pixelEventAux        | 65 |
| se-ort.bgc.t.requestBGC       | 65 |
| se.fastAxis                   | 65 |
| se.m.fastAxis                 | 65 |
| se.m.slowAxis                 | 65 |
| se.mot.m.motion               | 65 |
| se.mot.motion                 | 66 |
| se.mot.p.clocks               | 66 |
| se.mot.p.target               | 66 |
| se.mot.running                | 66 |
| se.mot.s.origin               | 66 |
| se.mot.s.origin.xOut          | 66 |
| se.mot.s.origin.yOut          | 66 |
| se.mot.s.running              | 66 |
| se.o.position                 | 67 |
| se.p.beginningOfFrame         | 67 |
| se.p.clocks                   | 67 |
| se.p.clocks.se.p.fastBlanking | 67 |
| se.p.closedLoop               | 67 |
| se.p.durationTrigger          | 67 |
| se.p.modes                    | 67 |
| se.p.modes.se.p.enable        | 67 |
| se.p.params                   | 68 |
| se.p.scanOffsets              | 68 |
| se.p.scanSize                 | 68 |
| se.p.transformMatrix          | 68 |
| se.p.x.fbParams               | 68 |
| se.p.y.fbParams               | 68 |
| se.s.blankingOff              | 68 |
| se.s.realXYOutputRate         | 68 |

|                                     |    |
|-------------------------------------|----|
| se.sid.sid.recordLateral            | 69 |
| se.sid.p.axisXNotY                  | 69 |
| se.sid.p.params                     | 69 |
| se.sid.prbs                         | 69 |
| se.sid.s.repetitions                | 69 |
| se.sid.s.running                    | 69 |
| se.sid.start                        | 69 |
| se.slowAxis                         | 69 |
| se.t.endOfFrame                     | 70 |
| se.t.endOfFrameTrigger              | 70 |
| se.t.endOfLine                      | 70 |
| se.t.endOfLine.Trig                 | 70 |
| se.t.outXYInt                       | 70 |
| se.tabs                             | 70 |
| sicm-flt.t.height                   | 70 |
| sicm-hdt.int.t.triggerCurve         | 70 |
| sicm-hdt.int.t.triggerCurveRelease  | 71 |
| sicm-hdt.t.triggerHeight            | 71 |
| sicm.Debug                          | 71 |
| sicm.blc.p.params                   | 71 |
| sicm.display                        | 71 |
| sicm.display.sicm.p.memorywWriteDec | 71 |
| sicm.m.upMotion                     | 71 |
| sicm.o.height                       | 71 |
| sicm.p.clocks                       | 72 |
| sicm.p.modes                        | 72 |
| sicm.p.params                       | 72 |
| sicm.s.baseline                     | 72 |
| sicm.upMotion                       | 72 |
| sid-hdt.int.t.idVar                 | 72 |
| sid.allowExternalRecord             | 72 |
| sid.m.motion                        | 72 |
| sid.o.count                         | 73 |

|                          |    |
|--------------------------|----|
| sid.p.idSource           | 73 |
| sid.p.params             | 73 |
| sid.p.start              | 73 |
| sid.prbs                 | 73 |
| sid.s.running            | 73 |
| sid.t2h.processVariable  | 73 |
| smd.address              | 73 |
| smd.p.writeMem           | 74 |
| smd.stepLUT              | 74 |
| stepmot.delay(ms)        | 74 |
| stepmot.dir              | 74 |
| stepmot.enable           | 74 |
| stepmot.on               | 74 |
| stepmot.running          | 74 |
| stepmot.steps            | 74 |
| wfm.Parameters           | 75 |
| wfm.Parameters.target    | 75 |
| wfm.displacementFinished | 75 |
| wfm.initialPositions     | 75 |
| wfm.microStepIndex       | 75 |
| wfm.rampIndex            | 75 |
| wfm.rampIndex.NextRamp   | 75 |
| wfm.rampRange            | 75 |
| wfm.reset                | 76 |

## Introduction

This is the introduction.

# System Architecture: Overview

This is the overview chapter.

## System Architecture: AFM Head and Scanner

This is the AFM Head and Scanner chapter.

### AFM Head

This is the AFM Head section.

### AFM Scanner

This is the AFM Scanner section.

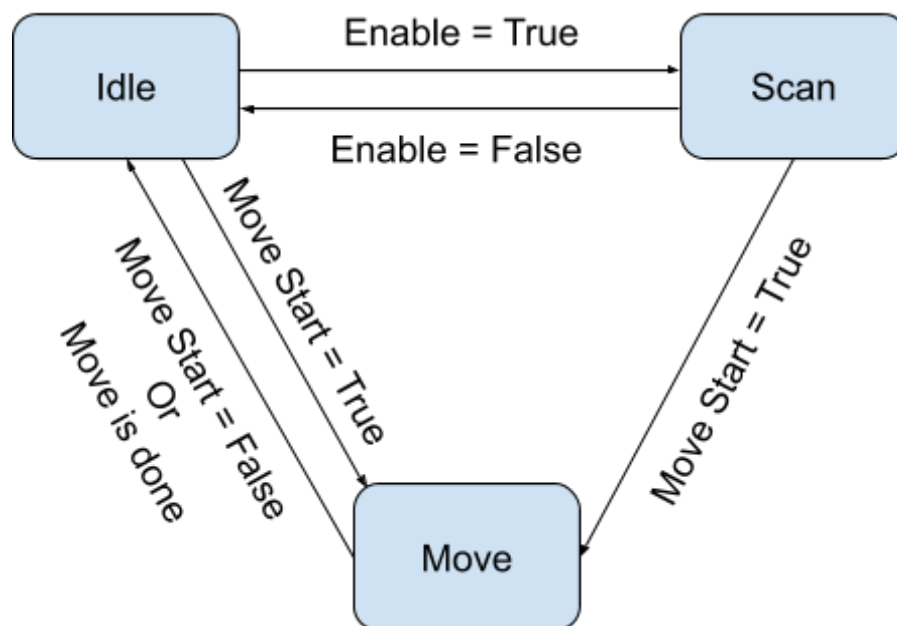
## System Architecture: Controller

This is the Controller chapter.

### X-Y Scanner

This loop generates two signals to control two axis scanner. Host sends the scanning parameters and scan engine (se\_main.vi) linearly interpolates the next positions for slow and fast axis. Beside that, scan engine counts down and updates triggers for end of line, end of frame and pixel event.

The X-Y Scanner - also called Scan Engine - can be seen as a state machine which includes three main modes of operation: Idle, Scan and Move. The Idle mode is where the engine does nothing but check for user inputs when the system is not active. The Scan mode is the usual mode of operation, where AFM tip's X-Y positions are calculated continuously in order to measure an image. Finally, the Move mode is used to move to a given X-Y destination (for example, back to the scan's origin).



Within each of these modes exist several states, either monitoring the current inputs and internal data of the scan engine or preparing data for outputs.

The scan engine's state machine also reacts to user inputs from the Host PC, allowing the user to change parameters as well as change the mode of operation. For example, modes can be switched based on two inputs: Enable (se.p.modes.[se.p.enable](#)) and Move Start (se.p.modes.se.mot.p.start). The state machine also acts on multiple outputs, mainly generating the X-Y position to be applied to the AFM Scanner at a given time interval as well as all the event synchronisation signals required by other parts of the system, such as a Pixel or End of Line events.

One important aspect regarding the implementation of these outputs by the state machine is guaranteeing that the updated X-Y position and the generation of events happen at the same time, without delays between each other. This means that the multiple processes of the system that manage control, outputs and measurements are correctly synchronised and ensures that measurements will be taken at the correct time.

The X-Y Scanner is implemented as the Scan Engine subVI [se\\_main\\_v2.vi](#) in [loop\\_042](#) of the FPGA code.

## Z Closed loop

This is the Z Closed loop section.

## Data Acquisition & Excitation

This is the Data Acquisition & Excitation section.

## Host Data Averaging

This is the Host Data Averaging section.

## Coarse Z

This is the Coarse Z section.

## System Architecture: Host PC

This is the Host PC chapter.

### Z Controller

This is the Z Controller section.

### Scan Parameters

This is the Scan Parameters section.

### Excitation

This is the Excitation section.

### Monitor

This is the Monitor section.

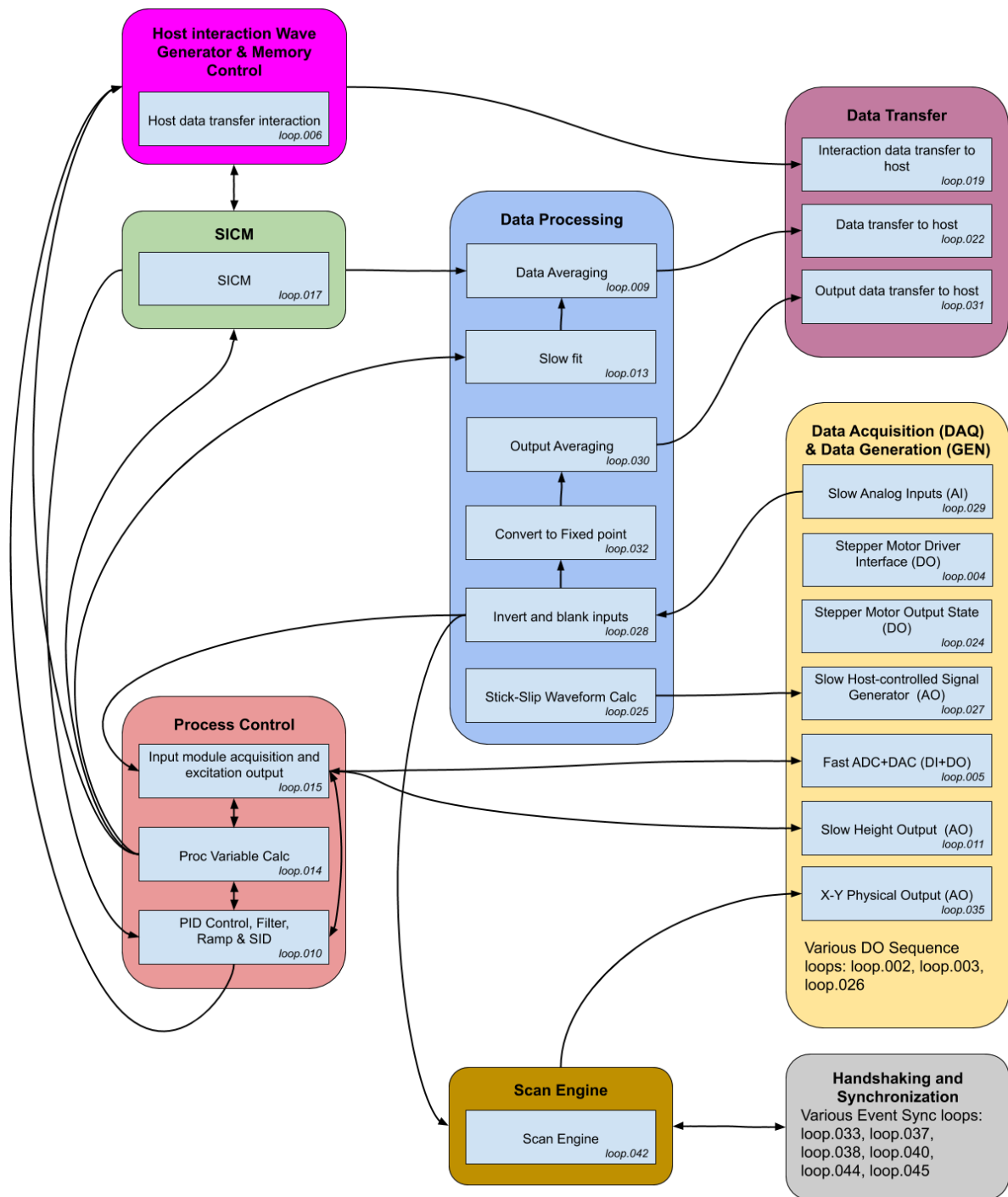
## Implementation: Overview

The FPGA code of the IbniAFMController contains many individual processes that perform different tasks, from Process Control to Data Acquisition. Some of these processes can be categorised into the following groups:

- **Process Control:** processes that perform tasks related to closed loop control and system identification
- **Data Processing:** processes which modify or convert the data for other processes, such as averaging, filtering and converting data types.
- **Data Acquisition (DAQ) and Generation (GEN):** processes which interact directly with the physical inputs and outputs of the system, usually comprised of analog and digital signals. These allow the system to interact with the physical world.
- **Data Transfer:** processes which send data in buffered way (i.e. without loss, assuming bandwidth is sufficient) to the host computer.

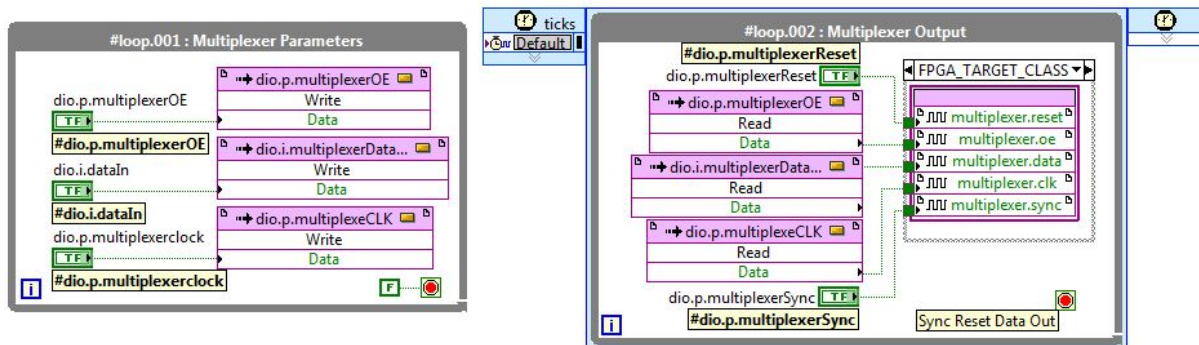
There are some processes which are not categorised, since they are specialised functions for this application, such the Scan Engine or SICM management.

The figure below is a schematic providing a general overview of the various groups and how the main processes in each interact with other processes from other groups. Note that most interactions shown are FIFOs. Additionally, many support processes are not depicted in this overview. These support processes are used to control other loops in short sequences, populating FPGA Registers or Memories, generating FPGA Interrupts.



## Implementation: Processes and loops

This section describes all the loops found in the FPGA top-level Main VI (controllerFPGA\_multiTarget.vi). Each loop is numbered and should ideally have a name, in order to quickly identify and understand its purpose. Some loops are standard While loops while others are Single-Cycle Timed Loops (SCTLs).



### loop.001 : Multiplexer Parameters

A while loop that takes three (of the five) input parameters for controlling the multiplexer that are on the front panel of the FPGA main and stores them in registers. Note that there are some discrepancies between the names of the corresponding controls and registers.

| Front Panel Control Input | Register Output         |
|---------------------------|-------------------------|
| dio.p.multiplexerOE       | dio.p.multiplexerOE     |
| dio.i.dataIn              | dio.i.multiplexerDataIn |
| dio.p.multiplexerclock    | dio.p.multiplexerCLK    |

Currently, these registers are only read in a single place, which is in the Multiplexer Output SCTL **loop.002**, potentially making this loop superfluous. In other words, this loop could likely be removed and the front panel controls could be put directly into **loop.002** without any negative side effects, although this would require testing.

### loop.002 : Multiplexer Output

A SCTL that takes all of the five input parameters for controlling the multiplexer (some from front panel controls, some from registers populated in **loop.001**) and applies them to the digital output pins. This SCTL runs at the Default top-level clock (usually 40MHz).

| Input                  | Input Type          | Output Pin        |
|------------------------|---------------------|-------------------|
| dio.p.multiplexerReset | Front Panel Control | multiplexer.reset |
| dio.p.multiplexerOE    | Register            | multiplexer.oe    |

|                         |                     |                  |
|-------------------------|---------------------|------------------|
| dio.i.multiplexerDataIn | Register            | multiplexer.data |
| dio.p.multiplexerCLK    | Register            | multiplexer.clk  |
| dio.p.multiplexerSync   | Front Panel Control | multiplexer.sync |

The three registers could be replaced with their corresponding front panel controls, as described in the **loop.001** section.

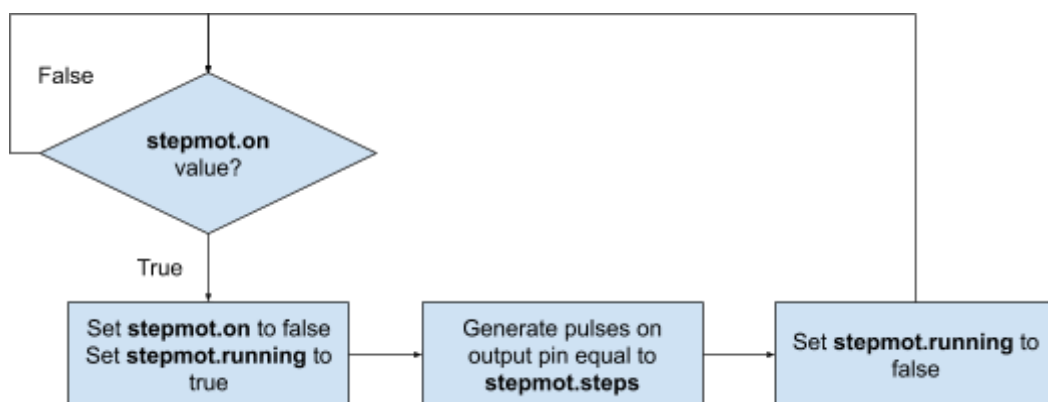
### loop.003 : Step Motor Enable & Direction Output

A SCTL that two of the five input parameters for controlling the stepper motor. This SCTL runs at the Default top-level clock (usually 40MHz).

| Front Panel Control Input | Output Pin        |
|---------------------------|-------------------|
| stepmot.enable            | stepmot.enable    |
| stepmot.dir               | stepmot.direction |

### loop.004 : Stepper Motor Driver Interface

A while loop that controls the stepper motor to perform a defined number of steps based on two Front Panel Controls (**stepmot.on** and **stepmot.steps**), by using a counter subVI **stateCounter\_v2.vi** which act on the **stepmot.steps** digital output pin. When the **stepmot.on** boolean Control becomes true, the counter starts counting steps and generates for each step a pulse on the output pin **stepmot.steps**, with the pulse's both high and low duration equal to the value of **stepmot.delay(ms)**. During this time, the **stepmot.running** indicator is set to true, so that the host can monitor when defined number of steps have been performed. When this operation has finished, the **stepmot.running** stays at false, until a new command is received on the **stepmot.on** control.



### loop.005 : Fast ADC acquisition and DAC generation

A SCTL that acquires fast 16-bit data from multiple digital input pins (ADC) and generates fast 16-bit data for multiple digital output pins (DAC) in a synchronous manner. There are two input parameters for this loop which define whether it is active and at what sampling (half-)time both the ADC and

DAC sections will run at. The input parameter **aio.o.fastInput** is a register which defines whether the ADC and DAC will run by deciding to generate (or not) a clock on output pin dedicated to the fast clock (depends on which target is being used). This register is populated in **loop.021** and is read in **loop.015** too. The other input parameter **aio.p.fastADCclockdec** defines the sampling rate of the ADC and DAC by defining the half-period of the clock. On each rising edge of the clock, data is acquired and generated on the defined digital I/O pins.

On top of acquiring and generating data, this loop communicates said data with **loop.015** using two FIFOs, one in each direction.

The loop sends acquired fast data through the **aio.t.fastVert** FIFO to **loop.015**, which will be used to construct indexed data (index-data pairs in a cluster) for other parts of the code to analyse acquired data as a time-series waveform.

The loop receives fast data to generate through the **aoi.t.fastDrive** FIFO from **loop.015** and outputs these values (bit by bit) to digital output pins. The values sent are the index within the waveform to be generated.

This SCTL runs at a fixed 40MHz, except for the USB 7856R target, for which it runs at 160MHz.

### loop.006 : Host data transfer interaction wave generator and memory control

A SCTL where waveform data selected from various sources is sent to the host through **loop.019** using the **hdt.int.staging** FIFO. Depending on the source of the data, it is either stored in the **hdt.int.storage** memory or sent directly to **loop.019**. The source selector is an enum is defined for the following channels of data:

| Source Selector                   |
|-----------------------------------|
| Mem ch1                           |
| Mem ch2                           |
| fixed # of proc Var points        |
| streaming proc Var                |
| system ID on USB-RIO, otherwise F |
| Interleaved memory ch1+ch2        |

| Data source FIFO            | Received from? | Method of transfer                                                                                                         |
|-----------------------------|----------------|----------------------------------------------------------------------------------------------------------------------------|
| ort-hdt.int.t.fcBgnd        | loop.014       | Stored in <b>hdt.int.storage</b> memory, then read from the memory before being sent to <b>hdt.int.staging</b> output FIFO |
| sicm-hdt.int.t.triggerCurve | loop.017       | Stored in <b>hdt.int.storage</b> memory,                                                                                   |

|                      |          |                                                                                   |
|----------------------|----------|-----------------------------------------------------------------------------------|
|                      |          | then read from the memory before being sent to <b>hdt.int.staging</b> output FIFO |
| fb-hdt.int.t.procVar | loop.014 | Sent to <b>hdt.int.staging</b> output FIFO                                        |
| sid-hdt.int.t.idVar  | loop.010 | Sent to <b>hdt.int.staging</b> output FIFO                                        |

This SCTL runs between 40MHz and 62.5MHz depending on the target where the code runs.

#### loop.007 : z Watchdog for auto-approach

A while loop that generates an IRQ to the host when the Z height is greater than a defined threshold and if this functionality is armed. This “Z height > threshold” comparison occurs in **loop.010**, where the Z height is compared to the **fb.p.zWatchDogTreshold** control. The result of this comparison is stored in the **fb.Z.WatchdogActive** register in **loop.010** and is read in the loop. If the function is armed with the **fb.zWatchdogArm** boolean control and if there is a rising edge on the **fb.z.WatchdogActive** register’s value, then the IRQ is performed. The IRQ used here is number 8, which is waited for in **ZController.vi** on the host.

#### loop.008 : Local wave recording hypervisor for ORT

A while loop that controls **loop.014** to perform local wave recording by using two FIFOs for handshaking. It uses the **lwr.t.start** FIFO to start a sequence and waits for the acknowledgement using the **lwr.t.done** FIFO. Once this short sequence has finished, an IRQ is generated.

The IRQ used here is number 7. It seems that currently, this Interrupt number is not being used on the host.

#### loop.009 : Host data averaging

A SCTL that calculates averages of data from multiple sources and then sends them to **loop.022** which will transfer the results to the host. This loop has 3 input parameters controls:

- **hdt.p.mainAverageEnabled** which defines whether averaging is active or not,
- **hdt.o.enable** which decides if the output FIFO outBundle is populated with new data or not
- **hdt.mainOutStreamSelect** which selects which category of data to stream to the host between PID/ORT and SICM.

The loop can average the following data channels, before sending them to **loop.022**:

| FIFO Name         | Category | Source Process                                 |
|-------------------|----------|------------------------------------------------|
| fb-hdt.t.height   | PID      | in loop.010                                    |
| fb-hdt.t.error    | PID      | in loop.010                                    |
| fb-hdt.t.bwdSlope | ORT      | in loop.013 -> ort_fit_calcSlopeDissipation.vi |
| fb-hdt.t.minBwd   | ORT      | in loop.014 -> ort_fit_calcIntermediate.vi     |

|                          |      |                                                |
|--------------------------|------|------------------------------------------------|
| fb-hdt.t.dissipation     | ORT  | in loop.013 -> ort_fit_calcSlopeDissipation.vi |
| outAuxIn                 | ORT? | in loop.014                                    |
| sicm-hdt.t.triggerHeight | SICM | in loop.017                                    |
| fb-sicm.t.current        | SICM | in loop.014                                    |

This SCTL runs at the Default top-level clock (usually 40MHz).

### loop.010 : PID calculation and height control

A SCTL that performs multiple tasks related to the control of the AFM tip: PID closed loop control (AFM and SICM modes), system identification using PRBS, filtering the control signal as well as generating ramp signals (in open loop).

There are input sources that are possible: PID, SICM and Ramp.

| Input | Input Name & Type                                      | Input Process |
|-------|--------------------------------------------------------|---------------|
| PID   | <b>fb.t.procVar</b> FIFO + <b>pid_main_v2.vi</b> subVI | in loop.014   |
| SICM  | <b>sicm-flt.t.height</b> FIFO                          | in loop.017   |
| Ramp  | <b>rmp_main.vi</b> subVI                               | <i>local</i>  |

The outputs of this loop are the error, the height and the output values, which are sent to other processes.

| Output       | Output Name        | Output Type | Output Process |
|--------------|--------------------|-------------|----------------|
| Error value  | fb-hdt.t.error     | FIFO        | in loop.009    |
| Height value | fb-hdt.t.height    | FIFO        | in loop.009    |
| Output value | fb-aio.t.heightInt | FIFO        | in loop.015    |

This SCTL runs between 40MHz and 62.5MHz depending on the target where the code runs.

### loop.011 : Slow Height generation (output)

A while loop that receives slow height drive data from a FIFO and applies them to two analog output pins. The input FIFO is **aio.t.slowHeightDrive**, which includes two values per element (in a cluster), which may be blanked (i.e. set to zero) or have their sign inverted individually, before being written on analog output pins (which depends on the target). The register **aio.o.invertOutputs** defines whether each of these values are inverted or not, and is populated in **loop.021** and read in **loop.035**.

This loop calls two instances of **invert\_and\_blank\_caseStruct.vi** in order to alter both values. These are placed in a SCTL (**loop.012**) in order to hasten execution of this part of the iteration. However, the speedup is extremely small, since the code inside the **invert\_and\_blank\_caseStruct.vi** subVI runs in 3 ticks if not in a SCTL, so the gain is (at most) two ticks of 40MHz. Note that the analog output pins can be updated at 1MS/s at maximum.

### loop.012 : SCTL to hasten loop.011 execution

A SCTL used for hastening the execution of **loop.011**, more specifically the **invert\_and\_blank\_caseStruct.vi** subVI. See the description of **loop.011** for more details.

This SCTL runs at the Default top-level clock (usually 40MHz).

### loop.013 : Slow fit calculation

A SCTL that fits a slope to the xxx data before sending the results to **loop.009** using the **fb-hdt.t.bwdSlope** and **fb-hdt.t.dissipation** FIFOs.

The input data required for calculating the slope all come from **loop.014**.

| Input FIFO              | Source Process                             |
|-------------------------|--------------------------------------------|
| ort.fit.accSums         | in loop.014 -> ort_fit_calcIntermediate.vi |
| ort.fit.t.integralParts | in loop.014 -> ort_fit_calcIntermediate.vi |

This SCTL runs at the 40MHz Onboard clock.

### loop.014 : Main process variable calculation

A SCTL.

This SCTL runs between 40MHz and 62.5MHz depending on the target where the code runs.

### loop.015 : Input module acquisition and excitation output

A SCTL.

### loop.016 : Populate memory items for feedback from host

A while loop that populates multiple Memory items by the host through Front Panel controls. Note that there are some discrepancies between the names of the corresponding controls and memory items.

|                           |               |                |
|---------------------------|---------------|----------------|
| Front Panel Control Input | Memory Output | Memory Read in |
|---------------------------|---------------|----------------|

|                        |                      |                                                                                                                                                      |
|------------------------|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| aio.m.drive            | aio.excitation       | in loop.015                                                                                                                                          |
| ort.fit.m.weights      | ort.fit.accWeights   | in loop.014 -> ort_fit_calcIntermediate.vi -> fitParamExtraction.vi                                                                                  |
| fb.mot.m.motion        | fb.mot.motion        | <i>nowhere?</i>                                                                                                                                      |
| ort.fit.m.coefficients | ort.fit.coefficients | in loop.013 -> ort_fit_calcSlopeDissipation.vi -> linearFitCalc.vi                                                                                   |
| sicm.m.upMotion        | sicm.upMotion        | <i>nowhere?</i>                                                                                                                                      |
| sid.m.motion           | sid.prbs             | in loop.010 -> se_main.vi -> sid_output_binaryMem.vi<br>in loop.042 -> se_sid_main.vi -> sid_output_binaryMem.vi<br><i>also written in loop.043?</i> |

### loop.017 : SICM

A SCTL.

### loop.018 : End of transmission acknowledge

A while loop that generates an IRQ to the host when an element exists in the **hdt.int.t.transferDone** FIFO. This FIFO is written to inside the **hdt\_int\_generator.vi** subVI in **loop.006**.

The IRQ used here is number 5, which is waited for in multiple VIs, including **Excitation.vi**, **ORTForceCurves.vi**, **Oscilloscope.vi** and **SiCM.vi** on the host.

### loop.019 : Transfer interaction data to host

A while loop that transfers data from the local **hdt.int.staging** FIFO to the host through the **hdt.t2h.interaction** FIFO. This loop also includes a **hdt.int.s.dataLost** status indicator, which can be reset using the **hdt.int.s.clearDataLost** Front Panel control.

The data in the **hdt.int.staging** FIFO comes from **loop.006**, where a multitude of various sources can be selected to be sent through this FIFO. This loop is the mechanism through which data from a selected source in **loop.006** can be sent to the host.

### loop.020 : Transfer background to host

A while loop that generates an IRQ to the host when the background measurement is completed, through a rising edge on the **ort.bgc.backgroundOK** register. This register is written to in the **ort\_bgc\_main.vi** subVI in **loop.014**. If there is a rising edge on this register's value, then the IRQ is performed.

The IRQ used here is number 4, which is waited for in **Excitation.vi** on the host.

### loop.021 : Population of many registers and AI voltage range

A while loop that takes many input parameters for some stores them in registers and others applies them to voltage ranges of Analog Input I/O. Below are the various registers being written to and where they are read:

| Front Panel Control Input and Register Output | Register Read in                        |
|-----------------------------------------------|-----------------------------------------|
| aio.o.invertOutputs                           | in loop.011<br>in loop.035              |
| aio.o.invertInputs                            | in loop.015<br>in loop.028              |
| aio.o.invertHeight                            | in loop.015                             |
| aio.o.invert2Inputs                           | in loop.015 (only on PXIE_7975R target) |
| ort.o.piezoMode                               | in loop.015                             |
| aio.o.disableExc                              | in loop.015                             |
| aio.o.bypassDither                            | in loop.015 (only on PXIE_7975R target) |
| aio.o.fastInput                               | in loop.005<br>in loop.015              |
| aio.o.heightLimits                            | in loop.015                             |

Most of these registers are used in **loop.015** to manage module acquisition and excitation output.

The **AI.voltage Range** Front Panel control is used to apply to all pertinent Analog Input voltage ranges for a given target, usual composed of 8 Analog Inputs on Connector0 or Connector1.

### loop.022 : Host data transfer main

A while loop that transfers data from the local **outBundle** FIFO to the host through the **hdt.t2h.mainStream** FIFO. This loop also includes a **hdt.o.overflow** status indicator, which currently is not latched, making it useless. Instead, like in **loop.019**, it should latch the overflow status and then have a mechanism to reset it through a Front Panel control.

The data in the **outBundle** FIFO comes from the **hdt\_main\_averageChannels.vi** subVI in **loop.009**, where a multiple channels of data are sent through this FIFO. This loop is the mechanism through which the data from these channels averaged in **loop.009** can be sent to the host.

### loop.023 : Retract on background correct to get non-interaction background

A while loop.

#### loop.024 : Output the Step Motor state

A while loop that reads the **smd.stepLUT** Memory at address equal to the **smd.address** Front Panel control in order to apply it to digital outputs A, B, C and D.

#### loop.025 : Write a Sawtooth Waveform to 3 FIFOs (Stick-Slip)

A SCTL.

This SCTL runs at the Default top-level clock (usually 40MHz).

#### loop.026 : Triggers End of Line / Frame

A while loop.

#### loop.027 : Slow host-controlled signal generator

A while loop.

#### loop.028 : Invert and blank inputs

A SCTL.

This SCTL runs at the Default top-level clock (usually 40MHz).

#### loop.029 : Slow Analog inputs

A while loop.

#### loop.030 : Output averaging and pixel synchronous output

A SCTL.

This SCTL runs at the Default top-level clock (usually 40MHz).

#### loop.031 : Output to host

A while loop.

#### loop.032 : Input recording and passover to averaging

A while loop.

#### loop.033 : Pixel Sync

A SCTL.

This SCTL runs at the Default top-level clock (usually 40MHz).

### loop.034 : Automatic background correct request

A SCTL.

This SCTL runs at a fixed 5MHz.

### loop.035 : X-Y Physcal Output

A while loop.

### loop.036 : SCTL to hasten loop.035 execution

A SCTL used for hastening the execution of **loop.035**, more specifically the **invert\_and\_blank\_caseStruct.vi** subVI. See the description of **loop.035** for more details.

This SCTL runs at the Default top-level clock (usually 40MHz).

### loop.037 : Pixel Events Aux Sync

A SCTL.

This SCTL runs at the Default top-level clock (usually 40MHz).

### loop.038 : Monitor Clear Pixel Event

A SCTL.

This SCTL runs at a fixed 62.5MHz.

### loop.039 : HSDC recording

A while loop that controls **loop.014** to perform HSDC recording by using two FIFOs for handshaking. It uses the **hxdc.t.start** FIFO to start a sequence and waits for the acknowledgement using the **hxdc.t.done** FIFO. Once this short sequence has finished, an IRQ is generated.

The IRQ used here is number 6. It seems that currently, this Interrupt number is not being used on the host.

### loop.040 : Monitor Pixel Sync Event

A SCTL.

This SCTL runs at the Default top-level clock (usually 40MHz).

### loop.041 : Read and write HSDC storage

A SCTL.

This SCTL runs at the Default top-level clock (usually 40MHz).

### loop.042 : Scan loop

A SCTL that includes the X-Y Scanner's Scan Engine ([se\\_main\\_v2.vi](#)), the System Identification for the Scan Engine ([se\\_sid\\_main.vi](#)) and PID control for both X and Y axes (2x [pid\\_XY.vi](#)). The goal of this loop is to generate the X-Y positions for the AFM Scanner through the [se.t.outXYInt](#) FIFO (going to [loop.035](#)) as well as generating all synchronisation signals necessary in other loops of the FPGA code. The PID closed loop control functionality uses the FIFO [aio-hdt.x](#) as an input, which comes from [loop.028](#). The Scan Engine uses data stored in three memories to be able to generate the X-Y positions: [se.fastAxis](#), [se.slowAxis](#) and [se.mot.Motion](#).

The inputs of this loop (and its SubVIs) are the following:

| Input Name           | Input Type          | Input Process    |
|----------------------|---------------------|------------------|
| se.mot.p.clocks      | Front Panel Control | <i>from Host</i> |
| se.p.clocks          | Front Panel Control | <i>from Host</i> |
| se.p.durationTrigger | Front Panel Control | <i>from Host</i> |
| se.p.modes           | Front Panel Control | <i>from Host</i> |
| se.p.scanOffsets     | Front Panel Control | <i>from Host</i> |
| se.p.transformMatrix | Front Panel Control | <i>from Host</i> |
| se.p.scanSize        | Front Panel Control | <i>from Host</i> |
| se.mot.p.target      | Front Panel Control | <i>from Host</i> |
| se.sid.p.params      | Front Panel Control | <i>from Host</i> |
| se.sid.p.axisXNotY   | Front Panel Control | <i>from Host</i> |
| se.sid.start         | Front Panel Control | <i>from Host</i> |
| se.p.closedLoop      | Front Panel Control | <i>from Host</i> |
| se.p.x.fbParams      | Front Panel Control | <i>from Host</i> |
| se.p.y.fbParams      | Front Panel Control | <i>from Host</i> |
| aio-hdt.x            | FIFO                | in loop.028      |
| se.fastAxis          | Memory              | in loop.043      |
| se.slowAxis          | Memory              | in loop.043      |
| se.mot.Motion        | Memory              | in loop.043      |

The outputs of this loop (and its SubVIs) are the following:

| Output Name              | Output Type           | Output Process            |
|--------------------------|-----------------------|---------------------------|
| se.s.blankingOff         | Front Panel Indicator | <i>to Host??</i>          |
| se.mot.s.running         | Front Panel Indicator | <i>to Host??</i>          |
| se.mot.s.origin          | Front Panel Indicator | <i>to Host??</i>          |
| se.o.position            | Front Panel Indicator | <i>to Host??</i>          |
| clocksOut                | Front Panel Indicator | <i>to Host??</i>          |
| se.sid.s.running         | Front Panel Indicator | <i>to Host??</i>          |
| se.sid.s.repetitions     | Front Panel Indicator | <i>to Host??</i>          |
| se.t.outXYInt            | FIFO                  | in loop.035               |
| se.mot.running           | Memory                | <i>nowhere??</i>          |
| se-hdt.t.pixelEvent      | FIFO                  | in loop.009 (or loop.037) |
| se-dio.t.pixelEvent      | FIFO                  | in loop.033               |
| se-hdt.t.pixelEventAux   | FIFO                  | in loop.030               |
| se.t.endOfLine           | FIFO                  | in loop.034               |
| se.t.endOfFrame          | FIFO                  | in loop.034               |
| hdt.t.mainStreamClear    | FIFO                  | in loop.009 (or loop.044) |
| se.p.beginningOfFrame    | FIFO                  | in loop.026               |
| se.t.endOfLine.Trig      | FIFO                  | in loop.026               |
| se.t.endOfFrameTrigger   | FIFO                  | in loop.026               |
| se.sid-sid.recordLateral | FIFO                  | in loop.010               |

This SCTL runs at the Default top-level clock (usually 40MHz).

#### loop.043 : Memory write access for Scan Engine

A while loop.

#### loop.044 : ITC Clear Pixel and Pixel Event Sync

A SCTL.

This SCTL runs at the Default top-level clock (usually 40MHz).

#### loop.045 : Monitor Pixel Sync Aux Event

A SCTL.

This SCTL runs at the Default top-level clock (usually 40MHz).

## Implementation: Functions and VIs

A description of the main functions that are used in the FPGA code.

[sicm\\_main.vi : SICM Main](#)

A subVI.

[ort\\_fit\\_calcSlopeDissipation.vi : Slow Fit](#)

A subVI.

[invert\\_and\\_blank\\_caseStruct.vi : Rock N Sock](#)

A subVI.

[ort\\_find\\_max.vi](#)

A subVI.

[ort\\_fit\\_start.vi](#)

A subVI.

[ort\\_fit\\_calcIntermediate.vi](#)

A subVI.

[ort\\_bgc\\_main.vi](#)

A subVI.

[circularCounter\\_v2.vi](#)

A subVI.

[inputFilter\\_aio.vi](#)

A subVI.

[lwr\\_main.vi : Inter Average to BRAM](#)

A subVI.

[intWaveMA.vi](#)

A subVI.

[pid\\_main\\_v2.vi](#)

A subVI.

[sid\\_main.vi](#)

A subVI.

[rmp\\_main.vi](#)

A subVI.

[flt\\_sos4xblock.vi](#)

A subVI.

[flt\\_sos8xblock.vi](#)

A subVI.

## hdt\_int\_generator.vi

A subVI.

## hdt\_main\_averageChannels.vi

A subVI.

## ort\_bgc\_hypervisor.vi

A subVI.

## lwr\_hypervisor.vi

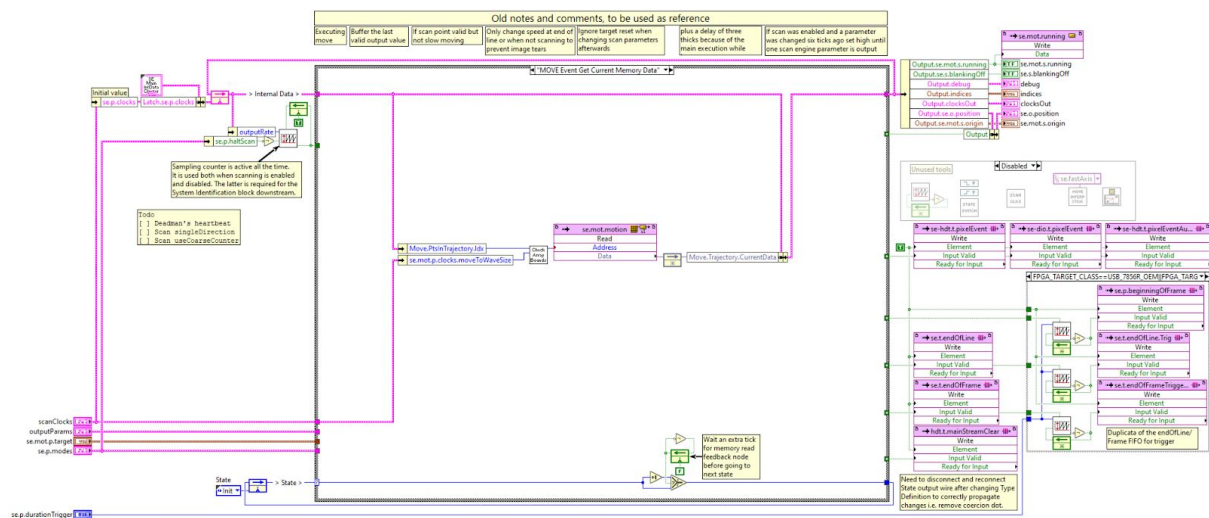
A subVI.

## hsdc\_hypervisor.vi

A subVI.

## se\_main\_v2.vi : Scan Engine Main (v2)

A subVI which implements the [X-Y Scanner](#) Scan Engine in the SCTL [loop.042](#), now on its 2nd version. The block diagram of this VI is composed of 3 parts: inputs and internal data on the left, state machine in the center and outputs to indicators and FIFOs to the right.

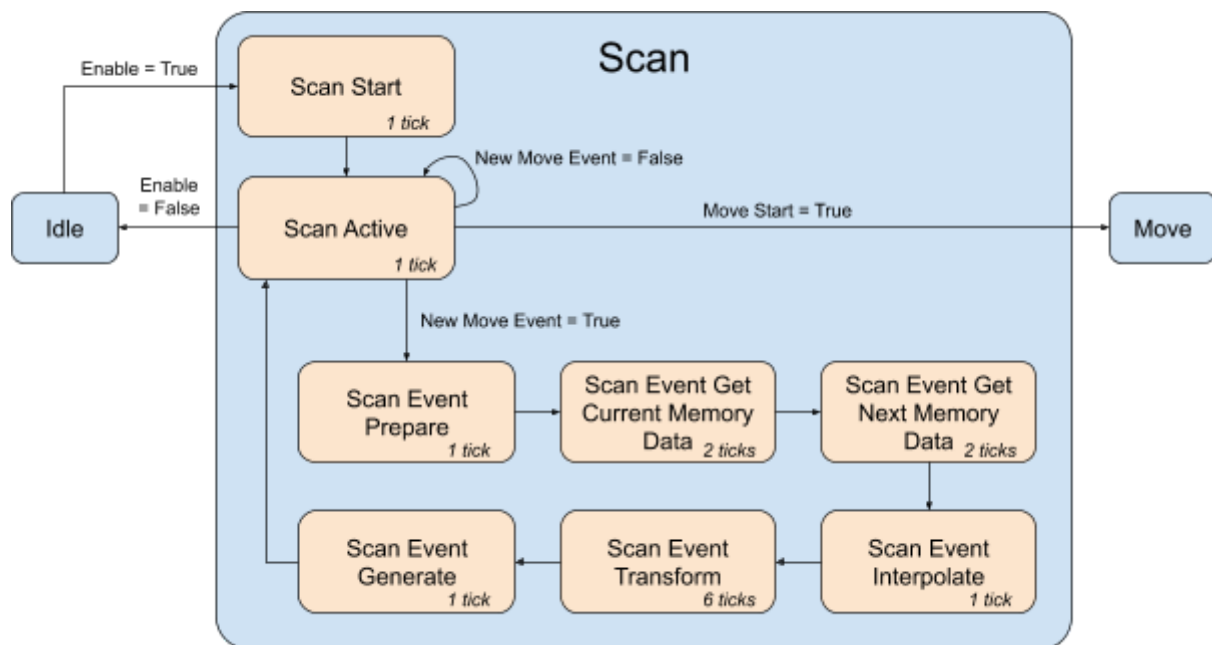


This state machine implementation attempts to fulfil several goals. Firstly, it must be readable, maintainable and scalable, even at the cost of performance (i.e. bandwidth and latency). Secondly, the main features of generating XY position and event synchronisation outputs must be retained, so as to not change the system's architecture and other processes. Thirdly, it should guarantee that the outputs are generated at the same time, without differences of latency between the different outputs and with consistent latency.

The result is a state machine programmed in this SCTL which contains 3 main operating modes: Idle, Scan and Move. These are described in the State enumeration as prefixes. The Idle state is relatively basic and waits for the Scan or Move modes to be enabled.

The Scan mode has its own "sub" state machine, as described in the diagram below. The state in which it remains most of the time is **Scan Active**, which monitors multiple inputs and internal variables to determine which event has to be generated next, if any. If no event is to be generated, then this state is repeated in the next cycle of the SCTL. If an event has to be generated, the state machine will perform several states in a row in a predetermined sequence during the upcoming

cycles of the SCTL. Each state repeats for a fixed number of cycles: some take a single cycle while others repeat for multiple ones. However, the output indicators and FIFOs are all updated only in the **Scan Event Generate** state. Once this state is performed, the state machine returns to the **Scan Active** state to wait for the next event to be generated. Note that this mechanism works only when the number of ticks between events is larger than the time it takes to perform this predetermined list of states. This implies that there is an upper bandwidth limit with this software implementation due to this of around 2.8MHz, because the sequence of states to be performed take 14 cycles at 40MHz.



When an event to be performed is determined by the **Scan Active** state, the result is kept until the **Scan Event Generate** state is performed. Based on the event, the appropriate outputs are updated and the internal data of the state machine is modified for the subsequent events to be appropriately selected and executed.

The list of scan events that can be generated as well as their descriptions are the following:

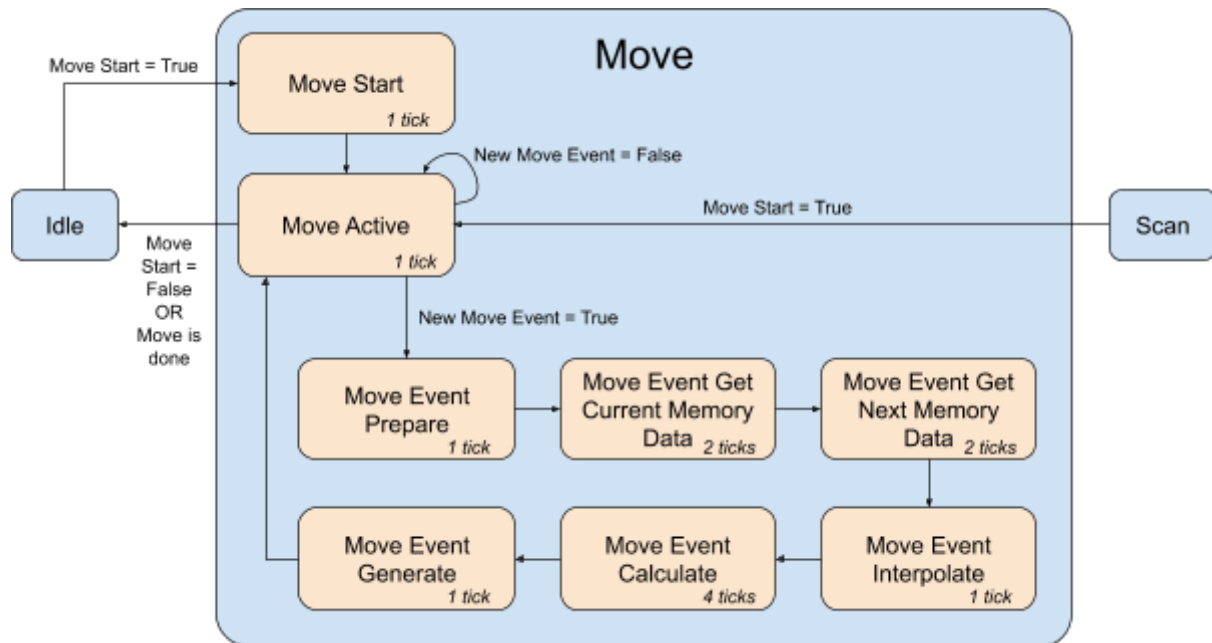
| Scan Event Name | Scan Event Description                                                                                                                                                                                                                                                                                 |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sample          | This event occurs when a new XY position needs to be generated to the output. Repeats every <b>se.p.clocks.outputRate</b> ticks.                                                                                                                                                                       |
| FastAx          | This event occurs when the interpolation index on the Fast Axis Waveform (stored in <b>se.fastAxis</b> ) must be incremented by 1. Repeats every <b>se.p.clocks.fastDecFactor</b> Sample events.                                                                                                       |
| Pixel           | This even occurs when a new pixel must be acquired by other processes. Repeats every <b>se.p.clocks.pplUpsampling</b> Sample events.                                                                                                                                                                   |
| FastAx+Pixel    | This event occurs when both FastAx and Pixel events occur on the same Sample event. This event is necessary because the FastAx and Pixel events are asynchronous between each other. In other words, the number of Pixels per line and the number of Fast Axis points per line are not related at all. |

|       |                                                                                                                                                                                                                                 |
|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Line  | This event occurs when the interpolation index on the Fast Axis Waveform (stored in <b>se.fastAxis</b> ) has reached the end of the Waveform and must be reset. Repeats every <b>se.p.clocks.outWaveSizeFast</b> FastAx events. |
| Frame | This event occurs when all the lines in a frame have been scanned. Repeats every <b>se.p.clocks.numLines</b> Line events.                                                                                                       |

The main outputs - the XY position and event synchronisation FIFOs - are updated as follows when in Scan mode:

| Output Name(s)                                                       | Output Type           | Scan State          | Scan Event                                                                                                              |
|----------------------------------------------------------------------|-----------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------|
| se.o.position                                                        | Front Panel Indicator | Scan Event Generate | Sample<br>FastAx<br>FastAx+Pixel<br>Line<br>Frame                                                                       |
| se-hdt.t.pixelEvent<br>se-dio.t.pixelEvent<br>se-hdt.t.pixelEventAux | FIFOs                 | Scan Event Generate | <b>If not in blanking zone:</b><br>Pixel<br>FastAx+Pixel<br>Line<br>Frame<br><b>If in blanking zone:</b><br><i>None</i> |
| se.t.endOfLine<br>se.t.endOfLine.Trig                                | FIFOs                 | Scan Event Generate | Line<br>Frame                                                                                                           |
| se.t.endOfFrame<br>se.t.endOfFrameTrigger                            | FIFOs                 | Scan Event Generate | Frame                                                                                                                   |
| se.p.beginningOfFrame                                                | FIFO                  | Scan Start          | <i>None</i>                                                                                                             |
| hdt.t.mainStreamClear                                                | FIFO                  | Scan Start          | <i>None</i>                                                                                                             |

In a similar way to the Scan mode, the Move mode also has its own “sub” state machine, as described in the diagram below. Other than the internal data and parameters, the main difference with the Scan mode is that once the Trajectory event has been generated, the state machine switches from the Move mode back to the Idle mode. Here also, there is an upper bandwidth limit with this software implementation of around 3.3MHz, because the sequence of states to be performed take 12 cycles at 40MHz.



The list of move events that can be generated as well as their descriptions are the following:

| Move Event Name | Move Event Description                                                                                                                                                                                                                                  |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sample          | This event occurs when a new XY position needs to be generated to the output. Repeats every <b>se.p.clocks.outputRate</b> ticks until the movement is done.                                                                                             |
| Point           | This event occurs when the interpolation index on the Movement Waveform (stored in <b>se.mot.motion</b> ) must be incremented by 1. Repeats every <b>se.mot.p.clocks.moveToDecFactor</b> Sample events until the movement is done.                      |
| Trajectory      | This event occurs when the interpolation index on the Movement Waveform (stored in <b>se.mot.motion</b> ) has reached the end of the Waveform. After this event has occurred, the movement is finished and the state machine switches to the Idle mode. |

The main outputs - the XY position and event synchronisation FIFOs - are updated as follows when in Move mode:

| Output Name(s)        | Output Type           | Move State          | Move Event                    |
|-----------------------|-----------------------|---------------------|-------------------------------|
| se.o.position         | Front Panel Indicator | Move Event Generate | Sample<br>Point<br>Trajectory |
| hdt.t.mainStreamClear | FIFO                  | Move Stop           | <i>None</i>                   |

Several implementation details remain to be carried over from Scan Engine v1 and implemented into Scan Engine v2, if deemed necessary. These include:

- “Deadman’s Hearbeat”: Sends a “fake” new position output for as long as the Scan Engine is in Idle mode every **se.p.clocks.outputRate** ticks. This is primarily used for the lateral system ID module after the scan signal as well as for system identification in [se\\_sid\\_main.vi](#).
- Single Direction: a toggle which allows the slow axis direction to alternate every frame or to always go in a single direction.
- Blanking with a coarse or precise counter.

### [se\\_sid\\_main.vi : Scan Engine System Identification Main](#)

A subVI.

### [pid\\_XY.vi](#)

A subVI.

### [hdt\\_aux\\_averageChannels.vi](#)

A subVI.

### [waveformGerneratorStickSlip\\_v3.vi](#)

NB: SubVI name is misspelt in the project.

A subVI.

## Implementation: Items

This list of items in the FPGA code includes:

- FPGA Nodes, such as FIFO, Register, Memory and Handshake read and write method nodes.
- Front Panel elements of the main FPGA VI controllerFPGA\_multiTarget.vi

### [AI.voltage](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [aio-hdt.aux](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [aio-hdt.x](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [aio-ort.t.endOfCycle](#)

Used in: controllerFPGA\_multiTarget.vi

Description: [Handshake between loops 14 \(main process calculation\) and 15 \(input acquisition\)](#). It indicates end of one ORT cycle calculated by counting down the number of points from [ort.p.waveSize](#). This handshake starts the calculation of the maximum of each cycle.

### [aio.excitation](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [aio.flt.bypass](#)

Used in: Filters.vi, controllerFPGA\_multiTarget.vi

Description:

### [aio.flt.p.anot4x](#)

Used in: Filters.vi, controllerFPGA\_multiTarget.vi

Description:

### [aio.flt.p.b4x](#)

Used in: Filters.vi, controllerFPGA\_multiTarget.vi

Description:

### [aio.i.samplingDecimation](#)

Used in: Oscilloscope.vi, ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### [aio.i.slowInputRate](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [aio.m.drive](#)

Used in: ExcitationParameters.vi, ExcitationParameters\_v2.vi, controllerFPGA\_multiTarget.vi

Description:

### [aio.o.bypassDither](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [aio.o.disableExc](#)

Used in: Excitation.vi, ORTForceCurves.vi, ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### [aio.o.fastInput](#)

Used in: Menu.vi, controllerFPGA\_multiTarget.vi

Description:

### [aio.o.heightLimits](#)

Used in: Filters.vi, controllerFPGA\_multiTarget.vi

Description:

### [aio.o.inputs](#)

Used in: SafeTip.vi, Scope.vit, Signals.vi, controllerFPGA\_multiTarget.vi

Description:

### [aio.o.invert2Inputs](#)

Used in: Analogs.vi, Calibration.vi, controllerFPGA\_multiTarget.vi

Description:

### [aio.o.invertHeight](#)

Used in: Analogs.vi, Calibration.vi, ForceCurveDecimated.vi, ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### [aio.o.invertInputs](#)

Used in: Analogs.vi, Calibration.vi, controllerFPGA\_multiTarget.vi

Description:

### [aio.o.invertOutputs](#)

Used in: Analogs.vi, Calibration.vi, controllerFPGA\_multiTarget.vi

Description:

### [aio.o.outputs](#)

Used in: AnalogOutputs.vi, Bias.vi, LaserOnOff.vi, SiCM.vi, Signals.vi, ZController.vi, controllerFPGA\_multiTarget.vi, dcMotor.vi

Description:

### [aio.p.fastADCclockdec](#)

Used in: Menu.vi, controllerFPGA\_multiTarget.vi

Description:

### [aio.s.HSaux](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

#### [aio.s.clock](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

#### [aio.s.fastHoriz](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

#### [aio.s.fastVertical](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

#### [aio.s.ortSubstDeflection](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

#### [aio.s.slowActualFBOutRate](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

#### [aio.s.slowActualInputRate](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

#### [aio.s.slowActualOutputRate](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

#### [aio.s.slowHoriz](#)

Used in: Photodiode.vi, SafeTip.vi, Scope.vit, Signals.vi, controllerFPGA\_multiTarget.vi

Description:

### [aio.s.slowVertical](#)

Used in: Photodiode.vi, SafeTip.vi, Scope.vit, SiCM.vi, Signals.vi, ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### [aio.s.sum](#)

Used in: Photodiode.vi, SafeTip.vi, Scope.vit, Signals.vi, controllerFPGA\_multiTarget.vi

Description:

### [aio.t.HSauxIn](#)

Used in: controllerFPGA\_multiTarget.vi

Description: FIFO used to store the data sampled by auxiliary channel (linked to AI?? channel). Data from this FIFO can be used as the feedback channel (control variable). Used in loop 14 (process value calc)

### [aio.t.fastDrive](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [aio.t.fastVert](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [aio.t.indexedValue](#)

Used in: controllerFPGA\_multiTarget.vi

Description: The FIFO that stores the points of one full ORT waveform. Each FIFO element stores two elements: one for the index of the datapoint of the waveform, and one for the value of the waveform.

### [aio.t.input](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [aio.t.slowHeightDrive](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [aio.t.slowHorizVert](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [aio.wfm.enable](#)

Used in: XYZStage.vi, controllerFPGA\_multiTarget.vi

Description:

### [aio.wfm.x](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [aio.wfm.y](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [aio.wfm.z](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [aoi.i.slowInputRate](#)

Used in: FGVfpgaAdress.vi, FGVfpgaAdressScanEngine.vi

Description:

### [clocksOut](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [dio.i.dataIn](#)

Used in: MultiplexerSendDataIn.vi, MultiplexerSerialInterfaceQueue.vi, controllerFPGA\_multiTarget.vi

Description:

### [dio.i.multiplexerDataIn](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [dio.p.multiplexeCLK](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [dio.p.multiplexerOE](#)

Used in: MultiplexerSendDataIn.vi, MultiplexerSerialInterfaceQueue.vi, controllerFPGA\_multiTarget.vi

Description:

### [dio.p.multiplexerReset](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [dio.p.multiplexerSync](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [dio.p.multiplexerclock](#)

Used in: MultiplexerSendDataIn.vi, MultiplexerSerialInterfaceQueue.vi, controllerFPGA\_multiTarget.vi

Description:

### [enableFeedback](#)

Used in: FeedbackParameters.vi

Description:

### fb-aio.t.heightInt

Used in: controllerFPGA\_multiTarget.vi

Description:

### fb-hdt.int.t.procVar

Used in: controllerFPGA\_multiTarget.vi, hdt\_int\_generator.vi

Description: ?? This FIFO contains the same data as fb.t.procVar (main process variable of the feedback loop). This is used to send data to host processed with a few other FIFOs in hdt-int-generator.vi ??

### fb-hdt.t.bwdSlope

Used in: hdt\_build\_stream\_ort.vi, ort\_fit\_calcSlopeDissipation.vi

Description:

### fb-hdt.t.dissipation

Used in: hdt\_build\_stream\_ort.vi, ort\_fit\_calcSlopeDissipation.vi

Description:

### fb-hdt.t.error

Used in: controllerFPGA\_multiTarget.vi, hdt\_build\_stream\_ort.vi

Description:

### fb-hdt.t.height

Used in: controllerFPGA\_multiTarget.vi, hdt\_build\_stream\_ort.vi

Description:

### fb-hdt.t.minBwd

Used in: hdt\_build\_stream\_ort.vi, ort\_fit\_calcIntermediate.vi

Description:

### fb-rmp.int.t.procVar

Used in: controllerFPGA\_multiTarget.vi, rmp\_main.vi

Description:

### [fb.sicm.t.current](#)

Used in: controllerFPGA\_multiTarget.vi, hdt\_build\_stream\_sicm.vi

Description:

### [fb.altSetpoint](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [fb.enable](#)

Used in: controllerFPGA\_multiTarget.vi, ort\_fit\_start.vi

Description:

### [fb.i.raw](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [fb.i.rescaled](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [fb.i.source](#)

Used in: Excitation.vi, Experiment.vi, FGVChannels.vi, Scan.vi, StateMachineInit.vi, ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### [fb.liftOff](#)

Used in: controllerFPGA\_multiTarget.vi, ort\_bgc\_hypervisor.vi

Description:

### [fb.loff.p.params](#)

Used in: Excitation.vi, StateMachineInit.vi, SyncParameters.vi, ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### [fb.mot.m.motion](#)

Used in: StateMachineInit.vi, controllerFPGA\_multiTarget.vi

Description:

### [fb.mot.motion](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [fb.mot.p.params](#)

Used in: Ramp.vi, StateMachineInit.vi, ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### [fb.mot.p.params.enableManualMotion](#)

Used in: ZController.vi

Description:

### [fb.mot.s.running](#)

Used in: Ramp.vi, ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### [fb.o.height](#)

Used in: ForceCurveSettings.vi, Ramp.vi, Scope.vit, Signals.vi, ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### [fb.p.allowSetpointSwitch](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### **fb.p.enable**

Used in: Excitation.vi, LAcontroller.vi, Ramp.vi, SiCM.vi, ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### **fb.p.limits**

Used in: Excitation.vi, StateMachineInit.vi, ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### **fb.p.limits.lowerLimit**

Used in: ZController.vi

Description:

### **fb.p.limits.upperLimit**

Used in: ZController.vi

Description:

### **fb.p.params**

Used in: PIDParameters.vi, controllerFPGA\_multiTarget.vi

Description:

### **fb.p.params.setPoint**

Used in: ForceCurveDecimated.vi

Description:

### **fb.p.zWatchDogTreshold**

Used in: ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### **fb.s.atLimit**

Used in: controllerFPGA\_multiTarget.vi

Description:

### fb.t.procVar

Used in: controllerFPGA\_multiTarget.vi

Description: FIFO used to store the value of the process variable for each feedback cycle. In contact mode, this is generated for each sample, but in ORT mode it is extracted from each cycle of the ORT waveform.

### fb.z.WatchdogActive

Used in: controllerFPGA\_multiTarget.vi

Description:

### fb.zWatchdogArm

Used in: ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### feedbackbParams

Used in: FeedbackParameters.vi

Description:

### flt.bypassPI

Used in: Ramp.vi

Description:

### flt.bypass1

Used in: LAcontroller.vi, StateMachineInit.vi, ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### flt.bypass2

Used in: Filters.vi, StateMachineInit.vi, ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### flt.bypassPI

Used in: Filters.vi, Ramp.vi, SiCMExperimentOnCheck.vi, SiCM.vi, ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### [flt.i.bypassSource](#)

Used in: Ramp.vi, SICMExperimentOnCheck.vi, SiCM.vi, ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### [flt.o.height](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [flt.o.hold](#)

Used in: LAcontroller.vi, controllerFPGA\_multiTarget.vi

Description:

### [flt.p.anot4x](#)

Used in: Filters.vi, controllerFPGA\_multiTarget.vi

Description:

### [flt.p.anot8x](#)

Used in: LAcontroller.vi, controllerFPGA\_multiTarget.vi

Description:

### [flt.p.b4x](#)

Used in: Filters.vi, controllerFPGA\_multiTarget.vi

Description:

### [flt.p.b8x](#)

Used in: LAcontroller.vi, controllerFPGA\_multiTarget.vi

Description:

### [flt.ss.p.enable](#)

Used in: Filters.vi, controllerFPGA\_multiTarget.vi

Description:

### [flt.ss.p.rate](#)

Used in: Filters.vi, controllerFPGA\_multiTarget.vi

Description:

### [hdt.aux.o.enable](#)

Used in: InitAndEmptyImageBuffer.vi, controllerFPGA\_multiTarget.vi

Description:

### [hdt.debugStream](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [hdt.int.p.memWriteSource](#)

Used in: Excitation.vi, ORTForceCurves.vi, SiCM.vi, controllerFPGA\_multiTarget.vi

Description:

### [hdt.int.p.readParams](#)

Used in: Excitation.vi, LAcontroller.vi, ORTForceCurves.vi, Oscilloscope.vi, SiCM.vi, controllerFPGA\_multiTarget.vi

Description: This clusters describes the parameters used for data transfer channel for non-critical data. It is used to send data that is not image data from target to the host. This kind of data is used for user information in the host VIs which are not time-critical such as ORT force curve or ORT average background, or transferring data for Oscilloscope.

### [hdt.int.p.readParams.hdt.int.p.readLength](#)

Used in: Oscilloscope.vi

Description:

### [hdt.int.p.readParams.hdt.int.s.transferSource](#)

Used in: ORTForceCurves.vi

Description:

### [hdt.int.s.clearDataLost](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

#### [hdt.int.s.dataLost](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

#### [hdt.int.s.debug](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

#### [hdt.int.staging](#)

Used in: controllerFPGA\_multiTarget.vi, hdt\_int\_generator.vi

Description:

#### [hdt.int.storage](#)

Used in: hdt\_int\_generator.vi

Description:

#### [hdt.int.t.transferDone](#)

Used in: controllerFPGA\_multiTarget.vi, hdt\_int\_generator.vi

Description:

#### [hdt.mainOutStreamSelect](#)

Used in: Excitation.vi, ORTForceCurves.vi, SiCM.vi, ZController.vi, controllerFPGA\_multiTarget.vi

Description:

#### [hdt.o.enable](#)

Used in: DataAcquisition.vi, DataAcquisitionAux.vi, InitAndEmptyImageBuffer.vi, controllerFPGA\_multiTarget.vi

Description:

#### [hdt.o.overflow](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

#### [hdt.o.ticksPerPixel](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

#### [hdt.outputEnable](#)

Used in: hdt\_main\_averageChannels.vi

Description:

#### [hdt.p.mainAverageEnabled](#)

Used in: DataAveraging.vi, controllerFPGA\_multiTarget.vi

Description:

#### [hdt.t.mainStreamClear](#)

Used in: controllerFPGA\_multiTarget.vi, hdt\_main\_averageChannels.vi, itc\_clearPixel.vi, rmp\_main.vi, se\_main.vi

Description:

#### [hdt.t2h.auxStream](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

#### [hdt.t2h.interaction](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

#### [hdt.t2h.mainStream](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

#### [hsdc.o.address](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

[hxdc.o.read](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

[hxdc.p.points](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

[hxdc.p.start](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

[hxdc.p.substBgnd](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

[hxdc.running](#)

Used in: hxdc\_record.vi

Description:

[hxdc.storage](#)

Used in: controllerFPGA\_multiTarget.vi, hxdcWriteToDRAM\_7975.vi

Description:

[hxdc.t.done](#)

Used in: hxdc\_hypervisor.vi, hxdc\_record.vi

Description:

[hxdc.t.start](#)

Used in: hxdc\_hypervisor.vi, hxdc\_record.vi

Description:

### [inBundleAux](#)

Used in: controllerFPGA\_multiTarget.vi, hdt\_aux\_averageChannels.vi

Description:

### [itc-hdt.t.pixelEventAux](#)

Used in: controllerFPGA\_multiTarget.vi, hdt\_aux\_averageChannels.vi

Description:

### [itc.p.allowBGC](#)

Used in: Excitation.vi, controllerFPGA\_multiTarget.vi

Description:

### [its.p.allowBGC](#)

Used in: Excitation.vi

Description:

### [loop.001](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [loop.002](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [loop.003](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [loop.004](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.005

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.006

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.007

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.008

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.009

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.010

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.011

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.012

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.013

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.014

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.015

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.016

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.017

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.018

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.019

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.020

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.021

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.022

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.023

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.024

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.025

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.026

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.027

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.028

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.029

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.030

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.031

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.032

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.033

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.034

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.035

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.036

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.037

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.038

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.039

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.040

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.041

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.042

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.043

Used in: controllerFPGA\_multiTarget.vi

Description:

### loop.044

Used in: controllerFPGA\_multiTarget.vi

Description:

### [loop.045](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [lwr.o.readData](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [lwr.p.avgCnt](#)

Used in: Ramp.vi, controllerFPGA\_multiTarget.vi

Description:

### [lwr.p.readAddress](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [lwr.p.start](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [lwr.sum](#)

Used in: lwr\_main.vi

Description:

### [lwr.t.done](#)

Used in: controllerFPGA\_multiTarget.vi, lwr\_hypervisor.vi

Description:

### [lwr.t.start](#)

Used in: controllerFPGA\_multiTarget.vi, lwr\_hypervisor.vi

Description:

### [mMot.running](#)

Used in: controllerFPGA\_multiTarget.vi, lwr\_main.vi

Description:

### [moveToWave](#)

Used in: moveToGetPointsFromRAM.vi

Description:

### [ort-hdt.int.t.fcBgnd](#)

Used in: controllerFPGA\_multiTarget.vi, hdt\_int\_generator.vi

Description:

### [ort-hdt.int.t.fcBgndRelease](#)

Used in: controllerFPGA\_multiTarget.vi, hdt\_int\_generator.vi

Description:

### [ort-ort.fit.t.start](#)

Used in: ort\_fit\_calcIntermediate.vi, ort\_fit\_start.vi

Description:

### [ort.bgc.average](#)

Used in: backgroundLinearAverageToBRAM.vi, backgroundSubstraction.vi

Description: [Memory containing the waveform of the averaged background. Used for ORT background subtraction.](#)

### [ort.bgc.backgroundOK](#)

Used in: controllerFPGA\_multiTarget.vi, ort\_bgc\_main.vi

Description:

### [ort.bgc.p.bgCntP2](#)

Used in: SyncParameters.vi, controllerFPGA\_multiTarget.vi

Description:

### [ort.bgc.p.scannerTc](#)

Used in: Excitation.vi, controllerFPGA\_multiTarget.vi

Description:

### [ort.bgc.p.start](#)

Used in: Excitation.vi, SyncParameters.vi, controllerFPGA\_multiTarget.vi

Description:

### [ort.bgc.p.waitTicks](#)

Used in: Excitation.vi, SyncParameters.vi, controllerFPGA\_multiTarget.vi

Description:

### [ort.bgc.requestType](#)

Used in: Excitation.vi, controllerFPGA\_multiTarget.vi

Description:

### [ort.bgc.sum](#)

Used in: backgroundLinearAverageToBRAM.vi

Description: Memory used during determination of the background during background correction procedure. It is used to sum up a certain number (user defined in host front panel in Excitation.vi "Average Count") of curves before averaging. This memory is used as a helper in backgroundLinearAverageToBRAM.vi only.

### [ort.bgc.t.start](#)

Used in: ort\_bgc\_hypervisor.vi, ort\_bgc\_main.vi

Description:

### [ort.fit.accSums](#)

Used in: ort\_fit\_calcIntermediate.vi, ort\_fit\_calcSlopeDissipation.vi

Description:

### [ort.fit.accWeights](#)

Used in: controllerFPGA\_multiTarget.vi, fitParamExtraction.vi

Description:

### [ort.fit.coefficients](#)

Used in: controllerFPGA\_multiTarget.vi, linearFitCalc.vi

Description:

### [ort.fit.m.coefficients](#)

Used in: FitCoefficientsCalculationDynamicRange.vi, controllerFPGA\_multiTarget.vi

Description:

### [ort.fit.m.weights](#)

Used in: FitCoefficientsCalculationDynamicRange.vi, controllerFPGA\_multiTarget.vi

Description:

### [ort.fit.p.params](#)

Used in: ExcitationParameters.vi, ExcitationParameters\_v2.vi, SyncParameters.vi, controllerFPGA\_multiTarget.vi

Description:

### [ort.fit.t.integralParts](#)

Used in: ort\_fit\_calcIntermediate.vi, ort\_fit\_calcSlopeDissipation.vi

Description:

### [ort.fit.waveBwd](#)

Used in: controllerFPGA\_multiTarget.vi, fitParamExtraction.vi

Description:

### [ort.fit.waveFwd](#)

Used in: controllerFPGA\_multiTarget.vi, fitParamExtraction.vi

Description:

### [ort.interactionFiltered](#)

Used in: intWaveMA.vi

Description:

#### [ort.o.interactionMax](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

#### [ort.o.maxIndex](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

#### [ort.o.piezoMode](#)

Used in: Excitation.vi, controllerFPGA\_multiTarget.vi

Description:

#### [ort.p.fcAvg](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

#### [ort.p.fcAvgEnable](#)

Used in: SyncParameters.vi, controllerFPGA\_multiTarget.vi

Description:

#### [ort.p.methodParams](#)

Used in: Excitation.vi, SyncParameters.vi, ZController.vi, controllerFPGA\_multiTarget.vi

Description:

#### [ort.p.methodParams.ort.p.syncPoint](#)

Used in: Excitation.vi, ORTForceCurves.vi

Description:

#### [ort.p.methodParams.ort.p.useSyncPoint](#)

Used in: Excitation.vi

Description:

### [ort.p.methodParams.ort.p.winSizeP2](#)

Used in: ORTForceCurves.vi

Description:

### [ort.p.waveSize](#)

Used in: Excitation.vi, ExcitationParameters.vi, ExcitationParameters\_v2.vi, ForceCurveSettings.vi, ORTForceCurves.vi, StateMachineInit.vi, controllerFPGA\_multiTarget.vi

Description: [The number of points for each period of excitation; selected by user form the front panel.](#)

### [ort.waveSize](#)

Used in: controllerFPGA\_multiTarget.vi, ort\_fit\_calcIntermediate.vi

Description: [Memory block storing ort.p.waveSize](#)

### [outAuxIn](#)

Used in: controllerFPGA\_multiTarget.vi, hdt\_build\_stream\_ort.vi

Description: [This FIFO is a copy of FIFO: aio.t.HSauxIn to be used to be bundled with other data channels \(e.g. height, error,...\) and sent to host.](#)

### [outBundle](#)

Used in: controllerFPGA\_multiTarget.vi, hdt\_main\_averageChannels.vi

Description:

### [outBundleAux](#)

Used in: controllerFPGA\_multiTarget.vi, hdt\_aux\_averageChannels.vi

Description:

### [rmp-hdt.aux.t.pixelEvent](#)

Used in: hdt\_aux\_averageChannels.vi, itc\_collect\_auxPixelEvents.vi, rmp\_main.vi

Description:

### [rmp-hdt.t.pixelEvent](#)

Used in: hdt\_main\_averageChannels.vi, rmp\_main.vi

Description:

### [rmp.p.action](#)

Used in: Ramp.vi, controllerFPGA\_multiTarget.vi

Description:

### [rmp.p.params](#)

Used in: Ramp.vi, controllerFPGA\_multiTarget.vi

Description:

### [rmp.p.reset](#)

Used in: Ramp.vi, controllerFPGA\_multiTarget.vi

Description:

### [rmp.p.running](#)

Used in: Ramp.vi

Description:

### [rmp.p.triggerParams](#)

Used in: Ramp.vi, controllerFPGA\_multiTarget.vi

Description:

### [rmp.p.useIntHeightAsStart](#)

Used in: Ramp.vi, controllerFPGA\_multiTarget.vi

Description:

### [rmp.running](#)

Used in: rmp\_main.vi

Description:

### [rmp.s.running](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [se-dio.t.pixelEvent](#)

Used in: dio\_pixelSync.vi, se\_main.vi

Description:

### [se-hdt.t.pixelEvent](#)

Used in: controllerFPGA\_multiTarget.vi, hdt\_main\_averageChannels.vi, itc\_collect\_pixelEvents\_sctl.vi, se\_main.vi

Description:

### [se-hdt.t.pixelEventAux](#)

Used in: controllerFPGA\_multiTarget.vi, hdt\_aux\_averageChannels.vi, se\_main.vi

Description:

### [se-ort.bgc.t.requestBGC](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [se.fastAxis](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [se.m.fastAxis](#)

Used in: Hysteresis.vi, Scan.vi, ScanWaveforms.vi, StateMachineInit.vi, controllerFPGA\_multiTarget.vi

Description:

### [se.m.slowAxis](#)

Used in: Hysteresis.vi, Scan.vi, ScanWaveforms.vi, StateMachineInit.vi, controllerFPGA\_multiTarget.vi

Description:

### [se.mot.m.motion](#)

Used in: InitializeSmovement.vi, controllerFPGA\_multiTarget.vi

Description:

### [se.mot.motion](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [se.mot.p.clocks](#)

Used in: InitializeSmovement.vi, MoveToTarget.vi, controllerFPGA\_multiTarget.vi

Description:

### [se.mot.p.target](#)

Used in: MoveToTarget.vi, Scan.vi, ScanparametersToFPGA.vi, controllerFPGA\_multiTarget.vi

Description:

### [se.mot.running](#)

Used in: se\_main.vi

Description:

### [se.mot.s.origin](#)

Used in: TipPosition.vi, controllerFPGA\_multiTarget.vi

Description:

### [se.mot.s.origin.xOut](#)

Used in: Signals.vi

Description:

### [se.mot.s.origin.yOut](#)

Used in: Signals.vi

Description:

### [se.mot.s.running](#)

Used in: MoveToTarget.vi, controllerFPGA\_multiTarget.vi

Description:

### [se.o.position](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [se.p.beginningOfFrame](#)

Used in: controllerFPGA\_multiTarget.vi, se\_main.vi

Description:

### [se.p.clocks](#)

Used in: Scan.vi, ScanWaveforms.vi, ScanparametersToFPGA.vi, StateMachineInit.vi, controllerFPGA\_multiTarget.vi

Description:

### [se.p.clocks.se.p.fastBlanking](#)

Used in: ScanparametersToFPGA.vi

Description:

### [se.p.closedLoop](#)

Used in: CloseLoopScan.vi, controllerFPGA\_multiTarget.vi

Description:

### [se.p.durationTrigger](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [se.p.modes](#)

Used in: Hysteresis.vi, MoveToTarget.vi, Scan.vi, controllerFPGA\_multiTarget.vi

Description:

### [se.p.modes.se.p.enable](#)

Used in: DataAcquisition.vi, DataAcquisitionAux.vi, Scan.vi, UpdateBlockDataAux.vi

Description:

#### [se.p.params](#)

Used in: LAcontroller.vi

Description:

#### [se.p.scanOffsets](#)

Used in: ScanparametersToFPGA.vi, controllerFPGA\_multiTarget.vi

Description:

#### [se.p.scanSize](#)

Used in: Scan.vi, ScanparametersToFPGA.vi, controllerFPGA\_multiTarget.vi

Description:

#### [se.p.transformMatrix](#)

Used in: ScanparametersToFPGA.vi, controllerFPGA\_multiTarget.vi

Description:

#### [se.p.x.fbParams](#)

Used in: CloseLoopScan.vi, controllerFPGA\_multiTarget.vi

Description:

#### [se.p.y.fbParams](#)

Used in: CloseLoopScan.vi, controllerFPGA\_multiTarget.vi

Description:

#### [se.s.blankingOff](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

#### [se.s.realXYOutputRate](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [se.sid.sid.recordLateral](#)

Used in: se\_sid\_main.vi, sid\_main.vi

Description:

### [se.sid.p.axisXNotY](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [se.sid.p.params](#)

Used in: LAcontroller.vi, controllerFPGA\_multiTarget.vi

Description:

### [se.sid.prbs](#)

Used in: LAcontroller.vi, controllerFPGA\_multiTarget.vi

Description:

### [se.sid.s.repetitions](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [se.sid.s.running](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [se.sid.start](#)

Used in: LAcontroller.vi, controllerFPGA\_multiTarget.vi

Description:

### [se.slowAxis](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [se.t.endOfFrame](#)

Used in: controllerFPGA\_multiTarget.vi, se\_main.vi

Description:

### [se.t.endOfFrameTrigger](#)

Used in: controllerFPGA\_multiTarget.vi, se\_main.vi

Description:

### [se.t.endOfLine](#)

Used in: controllerFPGA\_multiTarget.vi, se\_main.vi

Description:

### [se.t.endOfLine.Trig](#)

Used in: controllerFPGA\_multiTarget.vi, se\_main.vi

Description:

### [se.t.outXYInt](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [se.tabs](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [sicm-flt.t.height](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [sicm-hdt.int.t.triggerCurve](#)

Used in: hdt\_int\_generator.vi, sicm\_main.vi

Description:

### [sicm-hdt.int.t.triggerCurveRelease](#)

Used in: hdt\_int\_generator.vi, sicm\_main.vi

Description:

### [sicm-hdt.t.triggerHeight](#)

Used in: SiCM.vi, controllerFPGA\_multiTarget.vi, hdt\_build\_stream\_sicm.vi

Description:

### [sicm.Debug](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [sicm.blc.p.params](#)

Used in: SiCM.vi, controllerFPGA\_multiTarget.vi

Description:

### [sicm.display](#)

Used in: SiCM.vi, controllerFPGA\_multiTarget.vi

Description:

### [sicm.display.sicm.p.memorywWriteDec](#)

Used in: SiCM.vi

Description:

### [sicm.m.upMotion](#)

Used in: SiCM.vi, controllerFPGA\_multiTarget.vi

Description:

### [sicm.o.height](#)

Used in: SiCM.vi, controllerFPGA\_multiTarget.vi

Description:

### [sicm.p.clocks](#)

Used in: SiCM.vi, controllerFPGA\_multiTarget.vi

Description:

### [sicm.p.modes](#)

Used in: SiCM.vi, ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### [sicm.p.params](#)

Used in: SiCM.vi, controllerFPGA\_multiTarget.vi

Description:

### [sicm.s.baseline](#)

Used in: SiCM.vi, controllerFPGA\_multiTarget.vi

Description:

### [sicm.upMotion](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [sid-hdt.int.t.idVar](#)

Used in: hdt\_int\_generator.vi, sid\_main.vi

Description:

### [sid.allowExternalRecord](#)

Used in: LAcontroller.vi, controllerFPGA\_multiTarget.vi

Description:

### [sid.m.motion](#)

Used in: LAcontroller.vi, controllerFPGA\_multiTarget.vi

Description:

### [sid.o.count](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [sid.p.idSource](#)

Used in: LAcontroller.vi, controllerFPGA\_multiTarget.vi

Description:

### [sid.p.params](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [sid.p.start](#)

Used in: LAcontroller.vi, controllerFPGA\_multiTarget.vi

Description:

### [sid.prbs](#)

Used in: controllerFPGA\_multiTarget.vi, sid\_output\_binaryMem.vi

Description:

### [sid.s.running](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [sid.t2h.processVariable](#)

Used in: sid\_main.vi

Description:

### [smd.address](#)

Used in: ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### [smd.p.writeMem](#)

Used in: StateMachineInit.vi, controllerFPGA\_multiTarget.vi

Description:

### [smd.stepLUT](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [stepmot.delay\(ms\)](#)

Used in: ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### [stepmot.dir](#)

Used in: ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### [stepmot.enable](#)

Used in: ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### [stepmot.on](#)

Used in: ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### [stepmot.running](#)

Used in: ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### [stepmot.steps](#)

Used in: ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### [wfm.Parameters](#)

Used in: XYZStage.vi, ZController.vi, controllerFPGA\_multiTarget.vi

Description:

### [wfm.Parameters.target](#)

Used in: XYZStage.vi, ZController.vi

Description:

### [wfm.displacementFinished](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [wfm.initialPositions](#)

Used in: XYZStage.vi, controllerFPGA\_multiTarget.vi

Description:

### [wfm.microStepIndex](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [wfm.rampIndex](#)

Used in: controllerFPGA\_multiTarget.vi

Description:

### [wfm.rampIndex.NextRamp](#)

Used in: XYZStage.vi, ZController.vi

Description:

### [wfm.rampRange](#)

Used in: XYZStage.vi, controllerFPGA\_multiTarget.vi

Description:

[wfm.reset](#)

Used in: XYZStage.vi, ZController.vi, controllerFPGA\_multiTarget.vi

Description: